

中山大学专业课《程序设计I》

第四节 循环结构

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

主要内容

1. while 循环
2. for 循环
3. do while 循环
4. break / continue 关键字
5. 短路，副作用，位运算
6. 程序调试



世界上第一位程序员 Ada Lovelace



奥古斯塔·爱达·金，勒芙蕾丝伯爵夫人

在1842年，Ada Lovelace 在朋友英国数学家、发明家兼机械工程师查尔斯·巴贝其发明的分析机上编写算法和程序，被称为“世界上第一个程序员”

结构化程序设计 (Structured Program Theorem)

□ **任何**复杂的程序逻辑，都可以只用**三种基本结构**（程序流）组合而成：

- 1. 顺序结构：**程序按照代码书写的先后顺序，自上而下逐条执行
- 2. 选择结构：**根据条件判断的结果，选择执行不同的代码路径
- 3. 循环结构：**在满足条件的情况下，重复执行一段代码

回顾

- ❑ 从键盘输入一个字符，判断该字符的类型（数字、大写字母、小写字母、其他类型）

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char c;
    scanf(" %c", &c);

    if ( isdigit(c) ) printf("数字\n");
    else if ( isupper(c) ) printf("大写字母\n");
    else if ( islower(c) ) printf("小写字母\n");
    else printf("其他类型\n");

    return 0;
}
```

每次判断都需要重新运行程序😅

while 循环

```
while(表达式)
{
    复合语句
}
```

```
while(表达式)
    简单语句
```

- ❑ 在while循环中，首先判断**循环条件表达式**是否为**真**(即给定的条件是否成立)，如果为真，就执行后面的语句/语句块（也称为**循环体**）
- ❑ 之后不断的**重复**上述过程，只要循环条件表达式**为真就执行**循环体
- ❑ 直到判断**循环条件表达式为假**，不再执行循环体，离开循环结构

while 循环

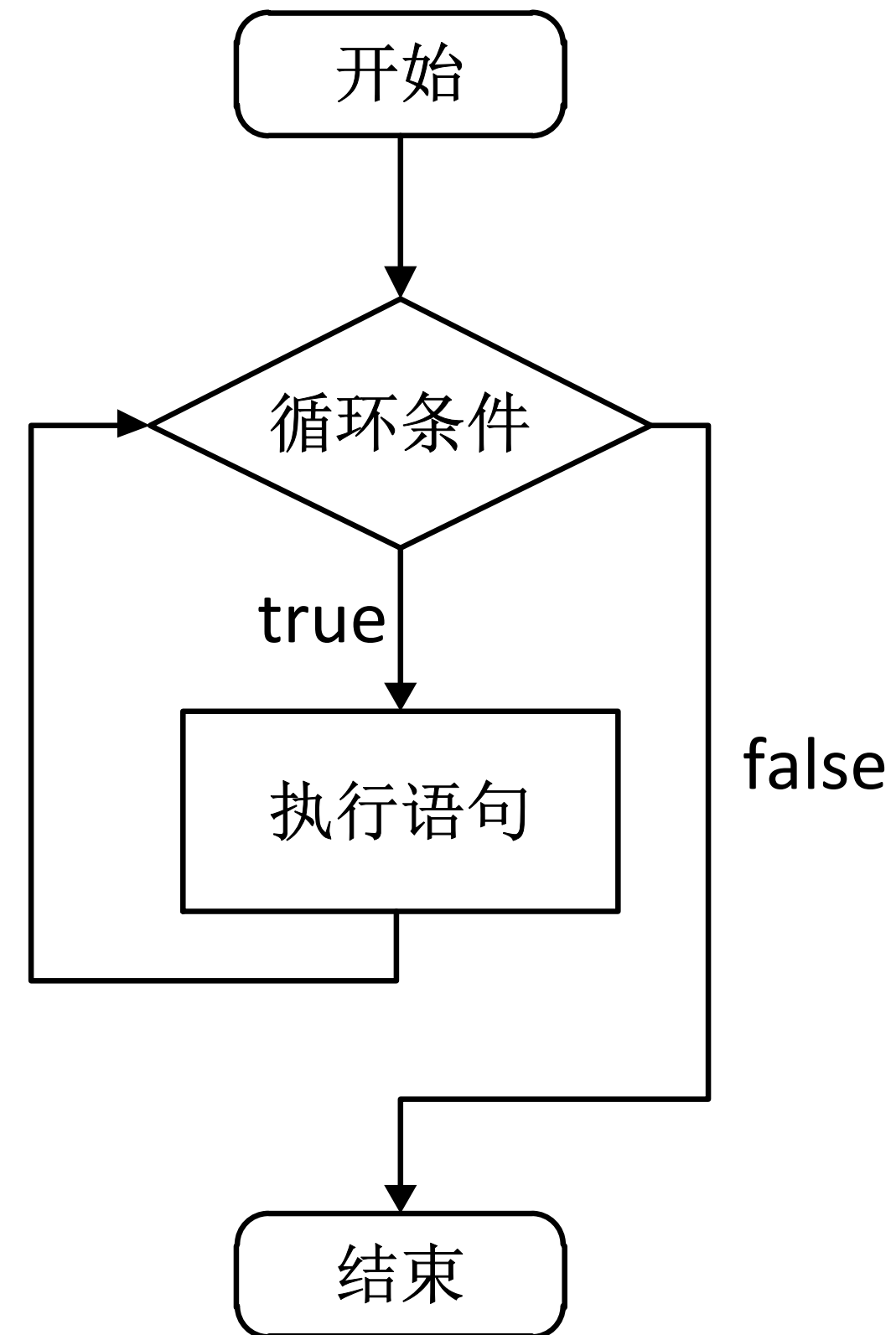
`while`(表达式)
语句

`if`(表达式)
语句

while 循环

```
while(表达式)
{
    复合语句
}
```

```
while(表达式)
    简单语句
```



练习

□ $1 + 2 + 3 + \dots + 100 = ?$

```
/*loop*/  
#include<stdio.h>  
  
int main(void) {  
    int sum = 0;  
    int i = 1;  
    while (i < 100)  
    {  
        sum += i++;  
    }  
    printf("The sum is %d",sum);  
}
```

死循环

□ $1 + 2 + 3 + \dots + 100 = ?$

```
#include<stdio.h>

int main(void) {
    int sum = 0;
    int i = 1;
    while (i < 100) {
        sum += i;
        i++;
    }
    printf("The sum is %d",sum);
}
```

```
#include<stdio.h>

int main(void) {
    int sum = 0;
    int i = 1;
    while (i < 100) {
        sum += i;
    }
    printf("The sum is %d",sum);
}
```

□ 循环终止条件永远无法满足，称为**死循环**

练习

□ 计算输入字符串的字符个数。

```
/*while count string*/
#include <stdio.h>

int main(void){
    int n=0;

    printf("Input a string:");
    while(getchar()!='\n') n++;

    printf("Number of characters: %d\n", n);
    return 0;
}
```

使用标志变量控制循环

```
/*while-add-learning*/
#include<stdio.h>
#include<stdbool.h>
#include<stdlib.h>
#include<time.h>

int main() {
    //give seed
    srand((unsigned)time(NULL)); //seed

    int i,j,sum;
```

```
    i = rand() % 100;
    j = rand() % 100;

    printf("%d + %d = ",i,j);
    scanf("%d",&sum);

    if (i + j == sum)
        printf("恭喜你，答对了！\n");
    else
        printf("Oooops，再努力一次。 \n");

    return 0;
}
```

使用标志变量控制循环

```
/*while-add-learning*/
#include<stdio.h>
#include<stdbool.h>
#include<stdlib.h>
#include<time.h>

int main() {
    //give seed
    srand((unsigned)time(NULL)); //seed

    int i,j,sum;
    bool flag = true; //标志变量
```

😞 回忆第2节scanf()相关内容

```
while (flag) {
    i = rand() % 100;
    j = rand() % 100;

    printf("%d + %d = ",i,j);
    scanf("%d",&sum);

    if (i + j == sum)
        printf("恭喜你，答对了！\n");
    else
        printf("Oooops，再努力一次。 \n");

    char c = getchar();
    if (c=='q' || c=='Q')
        flag = false;
}

return 0;
}
```

不确定循环与计数循环

计术循环：1-100求和

```
/*loop*/
#include<stdio.h>

int main(void) {
    int sum = 0;
    int i = 1;
    while (i < 100)
    {
        sum += i++;
    }
    printf("The sum is %d\n",sum);
}
```

不确定循环：计算字符数

```
/*while count string*/
#include <stdio.h>

int main(void){
    int n=0;

    printf("Input a string:");
    while(getchar()!='\n') n++;

    printf("Number of chars: %d\n", n);
    return 0;
}
```

for 循环

```
for(表达式1; 表达式2; 表达式3)  
    语句
```

表达式1：初始化

只执行一次，可以为零个、一个或多个变量设置初始值

表达式2：测试条件

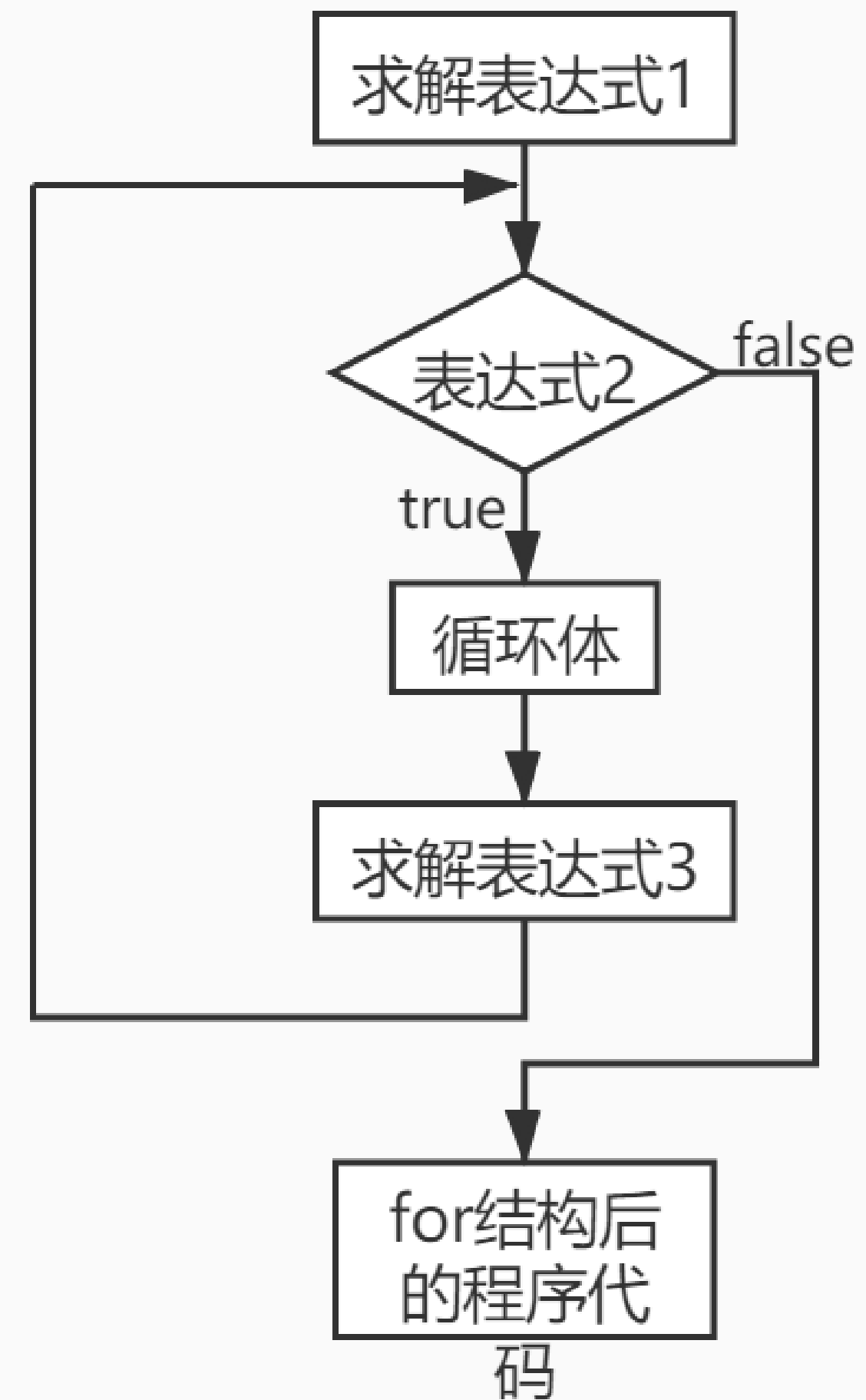
根据表达式真假决定是否继续执行循环

表达式3：修改循环变量

使循环变量增加，在循环结束时求值

for 循环

for(表达式1; 表达式2; 表达式3)
语句



for 循环的最常见形式

```
for(循环变量赋初值; 循环条件; 循环变量增值)
{
    语句块
}
```

表达式3要使循环变量向**趋于循环“结束”的方向**进行，避免死循环

for 循环与 while 循环的关系

```
int sum = 0;
for(int i = 1; i <= 10; i++)
{
    sum += i;
}
```

等价于

```
int i = 1, sum = 0;
while(i <= 10)
{
    sum += i;
    i++;
}
```

既然for 循环等价于 while 循环，那为什么需要 for 语句呢？

for 循环

```
int i = 1, sum = 0;  
while(i <= 10)  
{  
    sum += i;  
}
```

假设在循环体不存在修改循环变量的语句...😅

- ① 现实中有大量问题需要**枚举**有明确开始、结束数字或事物的序列
- ② 使用 for 语句符合自然语言语义，语法简明（开始，结束，增量）
- ③ 易于调试，不易出现死循环

for 循环：循环变量定义位置

```
/*for loop*/
#include<stdio.h>

int main() {
    int sum = 0;
    int i;
    for (i = 1; i < 100; i++) {
        sum += i;
    }
    printf("To %d, the sum is %d",i,sum);
}
```

```
/*for loop*/
#include<stdio.h>

int main() {
    int sum = 0;
    for (int i = 1; i < 100; i++) {
        sum += i;
    }
    printf("To %d, the sum is %d",i,sum);
}
```

for循环内定义的变量只有块作用域（仅限for循环）

for 循环

```
int sum = 0;
for(int i = 1; i <= 10; )
{
    sum += i;
    i++;
}
```

```
int sum = 0, i = 1;
for(; i <= 10; )
{
    sum += i;
    i++;
}
```

```
int i = 1, sum = 0;
while(i <= 10)
{
    sum += i;
    i++;
}
```

for 循环

```
int sum = 0;
for(int i = 1; ; i++)
{
    sum += i;
}
```

等价于

```
int i = 1, sum = 0;
while(1)
{
    sum += i;
    i++;
}
```

for 循环

3个表达式可以同时省略

```
for( ; ; )
```

```
while(1)
```

for 循环

“表达式1” 和 “表达式3” 可以是一个简单表达式也可以是**逗号表达式**

```
for( sum=0,i=1; i<=100; i++ ) sum=sum+i;
```

```
for( i=0,j=100; i<=100; i++,j-- ) k=i+j;
```

练习

□ 将以下程序改写成for循环

```
/*while count string*/
#include <stdio.h>

int main(void){
    int n=0;

    printf("Input a string:");
    while(getchar()!='\n') n++;

    printf("Number of characters: %d\n", n);
    return 0;
}
```

如何实现多次统计?

循环嵌套

```
while(表达式1)
{
    while(表达式2)
    {...}
}
```

```
for(表达式1.1;表达式1.2; 表达式1.3)
{
    for(表达式2.1;表达式2.2; 表达式2.3)
    {...}
}
```

一个循环体内又包含另一个完整的循环结构，称为**循环的嵌套**

循环嵌套

```
for(int i = 1; i <= 2; i++)  
{  
    for(int j = 1; j <= 3; j++)  
        printf("i = %d, j = %d, i*j = %d\n", i, j, i*j);  
}
```

循环嵌套

外循环



```
for(int i = 1; i <= 2; i++)  
{  
    for(int j = 1; j <= 3; j++)  
        printf("i = %d,j = %d,i*j = %d\n",i,j,i*j);  
}
```

循环嵌套

外循环



内循环



```
for(int i = 1; i <= 2; i++)  
{  
    for(int j = 1; j <= 3; j++)  
        printf("i = %d,j = %d,i*j = %d\n",i,j,i*j);  
}
```

内循环结束才会开始下一轮外循环

练习

□ 输出一个 4×4 的整数矩阵

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

练习

□ 输出一个 4×4 的整数矩阵

1	2	3	4
2	4	6	8
3	6	9	12
4	8	12	16

```
/*nested print matrix*/
#include <stdio.h>

int main(void)
{
    const int n = 4;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            printf("%-4d", i*j);
        }
        printf("\n");
    }
    return 0;
}
```

练习

□ 输出九九乘法表

1*1=1									
2*1=2	2*2=4								
3*1=3	3*2=6	3*3=9							
4*1=4	4*2=8	4*3=12	4*4=16						
5*1=5	5*2=10	5*3=15	5*4=20	5*5=25					
6*1=6	6*2=12	6*3=18	6*4=24	6*5=30	6*6=36				
7*1=7	7*2=14	7*3=21	7*4=28	7*5=35	7*6=42	7*7=49			
8*1=8	8*2=16	8*3=24	8*4=32	8*5=40	8*6=48	8*7=56	8*8=64		
9*1=9	9*2=18	9*3=27	9*4=36	9*5=45	9*6=54	9*7=63	9*8=72	9*9=81	

break/continue 语句

```
break;
```

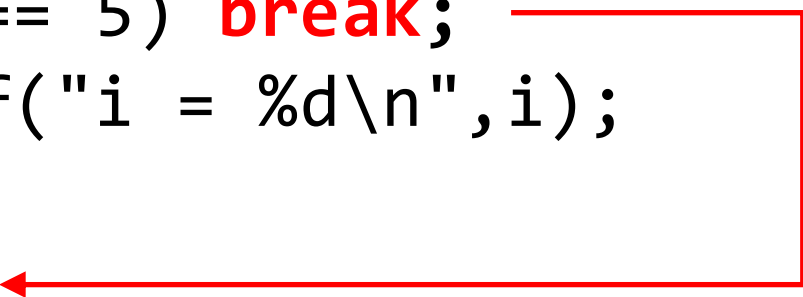
- ❑ break语句可以使当前流程跳出switch结构，继续执行switch结构后的下一条语句
- ❑ break语句还用来从循环体中**跳出整个循环结构**，即提前结束循环，执行循环结构后的下一条语句

```
continue;
```

- ❑ Continue语句可以提前**结束本次**循环，而接着执行下次循环

break/continue 语句

```
for(int i = 1;i <= 10; i++)  
{  
    if(i == 5) break;  
    printf("i = %d\n",i);  
}  
return 0;
```



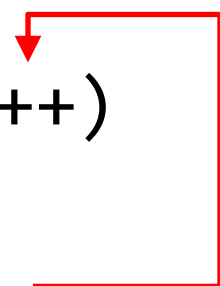
这个for循环结构的作用为打印数字1-10

第一个程序中:

当i = 5时, 满足if语句的判断, 执行break语句, 跳出整个循环, 循环结束

第二个程序中:

```
for(int i = 1;i <= 10; i++)  
{  
    if(i == 5) continue;  
    printf("i = %d\n",i);  
}  
return 0;
```



当i = 5时, 满足if语句的判断, 执行continue语句, 跳过本次循环中continue语句后面语句的执行, 转入下一次循环

练习：break与continue

- 按顺序打印 1–100 数，跳过 3 的倍数，当所有打印数字总和大于 n 时结束

do while 循环

□ 想要持续运行该如何修改程序？

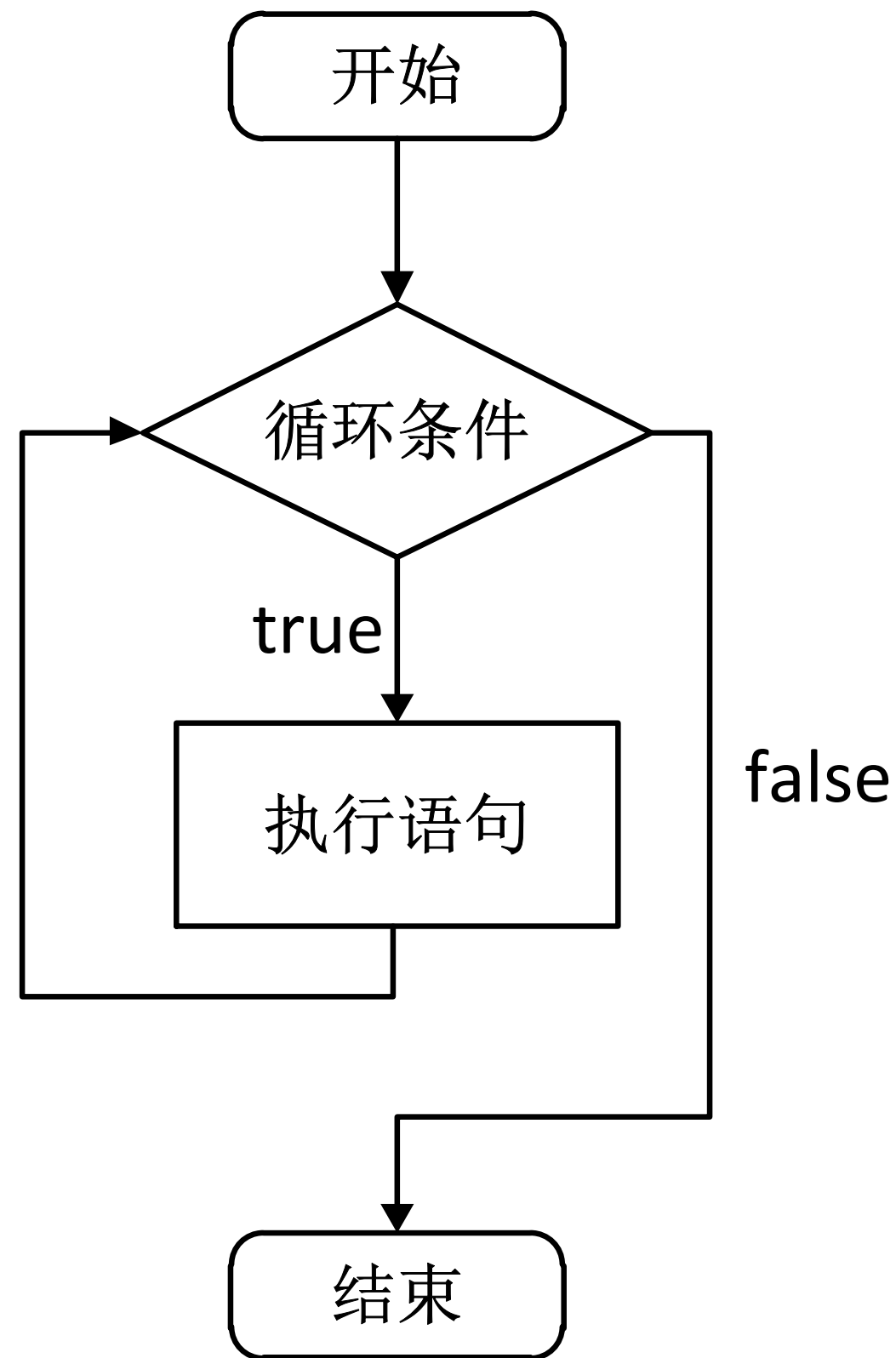
```
int n;  
scanf("%d", &n);  
  
for (int i=1,sum=0; i<=100; i++)  
{  
    if (i%3==0) continue;  
    sum += i;  
    if (sum>n) break;  
    printf("i=%-5d sum=%d\n", i, sum);  
}
```

do while 循环

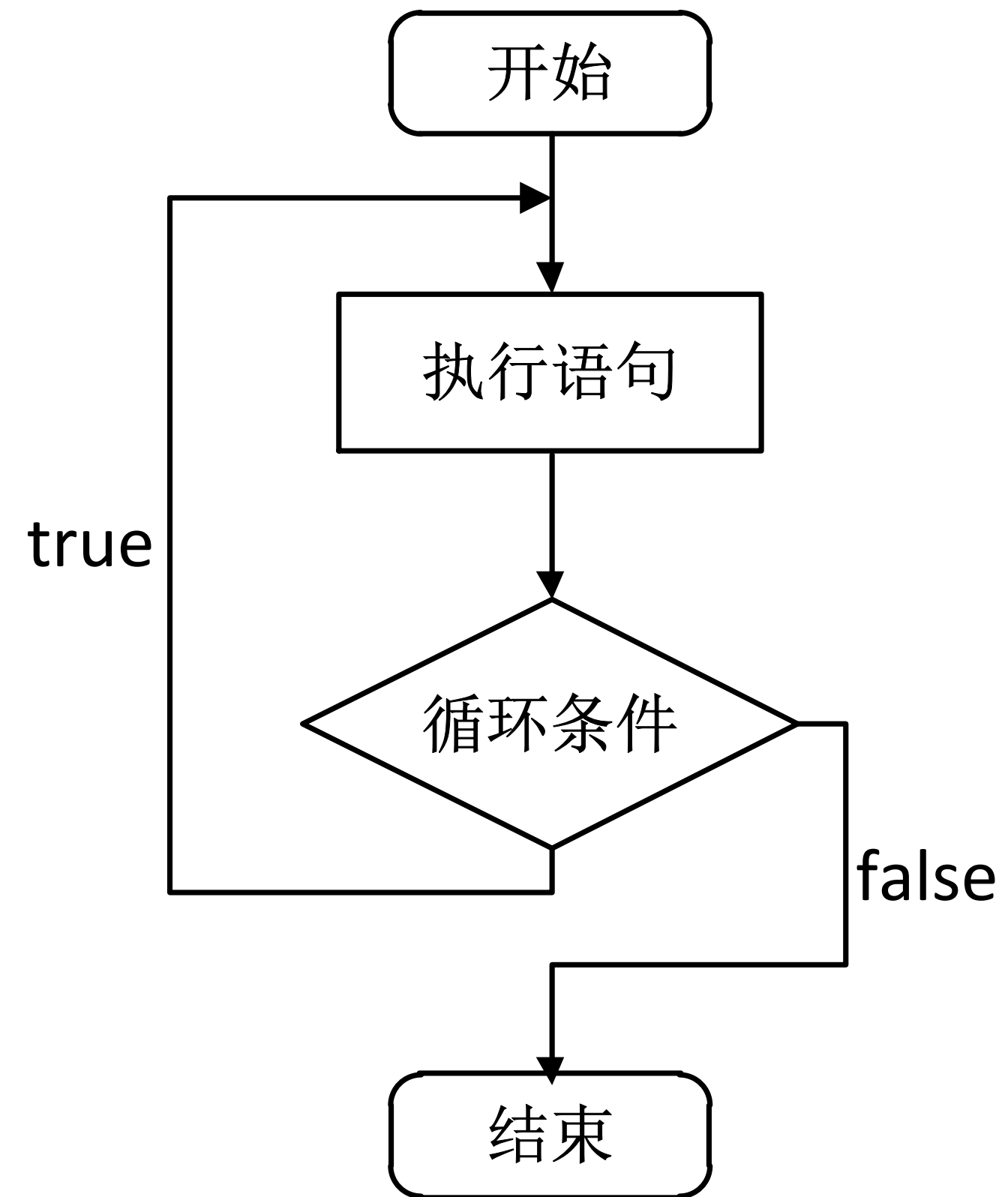
```
do  
{  
    语句块  
}  
while(表达式);
```

- ❑ **先执行一次**指定的循环体语句，然后判断表达式，当表达式的值为真时，返回重新执行循环体语句
- ❑ 如此反复，直到判断到表达式的值为假，结束循环

do while 循环



while



do while (至少执行一次)

猜数游戏

- 随机产生一个100以内的正整数，用户输入一个数进行猜测，编写程序自动与目标数进行比较，并提示大了（“Too big”），还是小了（“Too small”），相等表示猜到了（“Bingo”）

```
/*dowhile guess number*/
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

int main() {
    //give a number between 0 .. 99
    srand((unsigned)time(NULL)); //seed
    int target = rand() % 100;
}
```

主要内容

1. while 循环
2. for 循环
3. do while 循环
4. break / continue 关键字
5. 短路, 副作用, 位运算
6. 程序调试

主要内容

1. while 循环
2. for 循环
3. do while 循环
4. break / continue 关键字
- 5. 短路，副作用，位运算**
- 6. 程序调试**

思考

以下程序会输出什么？

```
#include <stdio.h>

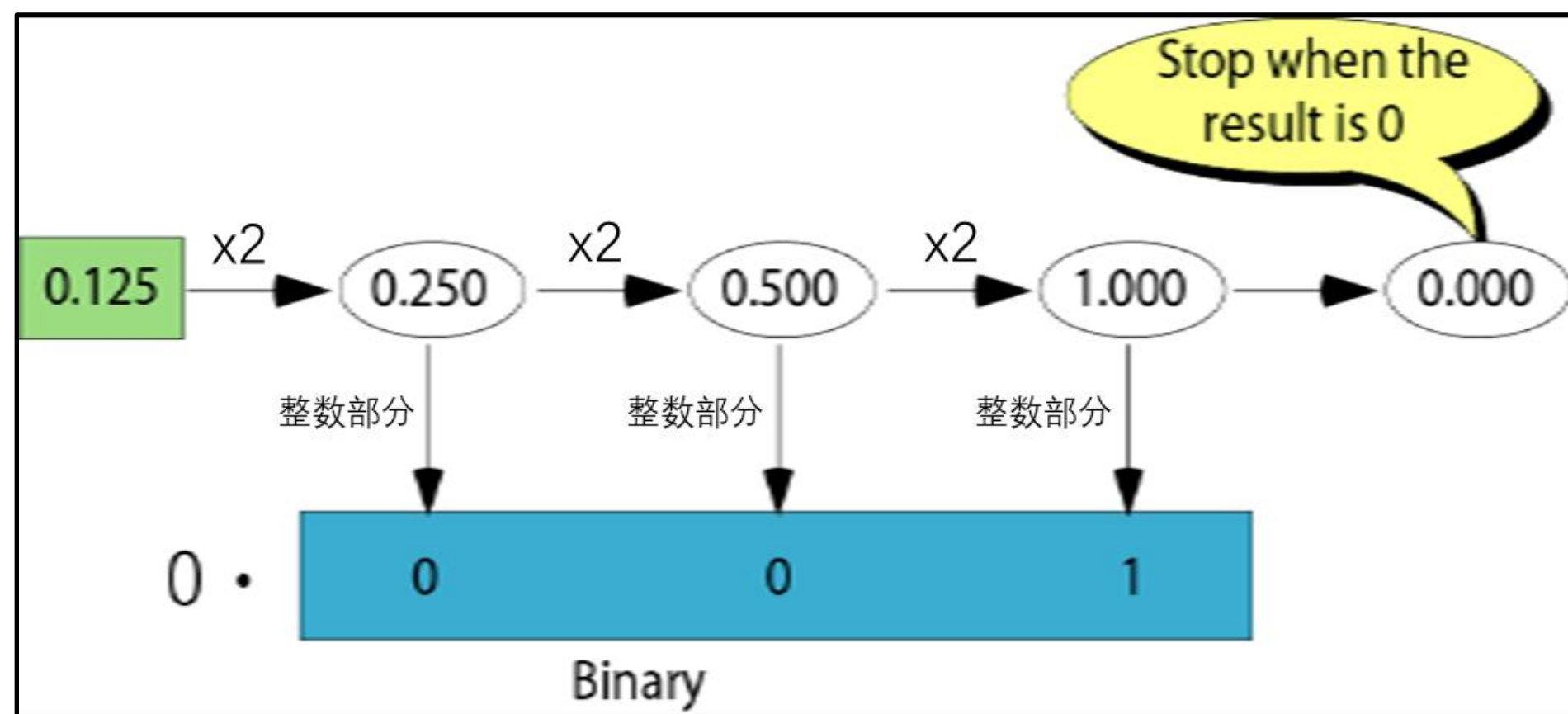
int main(void) {

    double x = 0.0;
    while (x != 1.0)
    {
        x += 0.1;
    }
    printf("x=%lf\n", x);

    return 0;
}
```

实数比较

- ❑ 计算机中float/double类型浮点数大多是实际数的**近似值**
- ❑ 回忆小数的二进制表示...



0.15
x2 0.30 -- 0
x2 0.60 -- 0
x2 1.20 -- 1 *
x2 0.40 -- 0 *
x2 0.80 -- 0 *
x2 1.60 -- 1 *
x2 1.20 -- 1 ...

$0.15 = (0.00100110011001...)_{2}$

实数比较

- ❑ 两个实数变量不能使用 `==` 或 `!=` 直接比较，或判断实数变量为零
 - 特别是循环语句若使用类似 `(x==y)` 这样的比较表达式，可能导致死循环或循环中止
- ❑ 实际中，实数比较运算一般仅使用 `>` 和 `<` 两个比较运算符
 - 判断实数相等使用 `fabs(x-y) < eps`
 - 判断实数等于零使用 `fabs(x) < eps`
 - `eps` 称为精度控制量（值），为合适的小数（如 `1e-7`）

练习

□ 计算 π ，精确到小数点后5位。 π 的计算公式如下：

$$\pi = 4 * (1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots + \frac{1}{4n-3} - \frac{1}{4n-1} + \dots)$$

```
double pi = 0;
int i = 1;
const double eps = 1e-6; // 精度保证5位
while ( 1.0/i > eps)      // 不合适用for循环
{
    pi += 1.0/i;
    i++;i++;
    pi -= 1.0/i;
    i++;i++;
}
pi *= 4;
printf("%.5f\n",pi);
```

思考

以下程序会输出什么？

```
int a,b,c,d;  
a = b = c = -1;  
d = ++a && ++b && ++c;  
printf("%d %d %d %d\n",a,b,c,d);
```

逻辑运算短路

□ 逻辑表达式 $a \ \&\& \ b$

- 若子表达式 a 为 false, 则子表达式 b 不求值, 结果必然为 false
- 这称为 **and 短路**

□ 表达式 $a \ || \ b$

- 若子表达式 a 为 true, 则子表达式 b 不求值, 结果必然为 true
- 这称为 **or 短路**

□ 可以优化程序执行速度

逻辑运算短路

□ 以下程序会输出什么？

```
int a,b,c,d;  
a = b = c = 0;  
d = a++ || ++b || c++;  
printf("%d %d %d %d\n",a,b,c,d);
```

□ 为提升程序性能：

- 在 and 表达式中把大概率为 false 的子表达式写在左边
- 在 or 表达式把大概率为 true 的子表达式写在左边

逻辑运算短路

□ 有 int 变量 a 和 b，请问下面的语句是什么意思？

```
result = b && (c=(double)a/b);
```



```
if (b == 0) result = false;  
else result = (_Bool)(c=(double)a/b);
```

□ 因此，逻辑运算短路等价于 if ... else if ... 多路分支语句

$e1 \ \&\& \ e2 \ \&\& \ e3 \ \&\& \ \dots \ \&\& \ en$

```
if (!e1) result=false;  
else if (!e2) result= false;  
else if (!e3) result= false;  
... ..  
else result= (_Bool)(en);
```

$e1 \ || \ e2 \ || \ e3 \ || \ \dots \ || \ en$

```
if (e1) result=true;  
else if (e2) result=true;  
else if (e3) result=true;  
... ..  
else result= (_Bool)(en);
```

副作用 (Side Effects)

- ❑ 如果一个表达式**不仅算出一个值**，还**修改了其他变量**，就说这个表达式有**副作用**，例如：自增减运算、赋值运算、传引用函数
- ❑ **有副作用的表达式并不一定有害**，但以下情况会导致表达式难以被理解、测试，甚至被不同编译器解释为不同语义的代码
 - **&&、||、?:** 表达式包含有副作用的子表达式。例如：**`a || i++`**
 - 子表达式之间**因副作用必须用特定顺序编译**，可能会导致**未定义**行为。例如：
`exp1 + exp2`不能认为`exp1`先执行，`exp2`再执行；如果`exp1`和`exp2`存在执行顺序约束，则结果是未定义
 - `i++ + i++`; `b*2 + 5*(1+b++)`; `a[i++] = i`; `f(&i) + i`;
 - 函数**调用参数**中包含有副作用表达式。例如；`f(c=d, e++)`

位掩码 (BitMask)

- 位掩码是用于操作数据中特定位的二进制数

```
A = 1010 1010
B = 0000 1111 (掩码)
-----
A&B = 0000 1010 屏蔽(清零)高4位
A|B = 1010 1111 置位低4位
A^B = 1010 0101 高4位直通, 低4位求反
```

位掩码 (BitMask)

□ & 可以**清零**指定的位

0	0	1	0	1	0	1	1
&	1	1	1	1	0	1	1
0	0	1	0	0	0	1	1
	↑				↑		
	不变				清零		

位掩码 (BitMask)

□ | 可以**设置**指定位为1

0	0	1	0	1	0	1	1
	1	1	1	1	0	0	0
1	1	1	1	1	0	1	1
		↑				↑	
		设置为1				不变	

位掩码 (BitMask)

□ ^ 可以**反转**指定位

$$\begin{array}{r} 00101011 \\ \wedge 11110000 \\ \hline 11011011 \end{array}$$

↑ 反转为1 ↑ 不变

位掩码应用案例

- 案例用 char 表示文件的权限，对应位为1表示有权限
 - 第一位表示可读，第二位表示可写，第三位表示可执行

```
#include <stdio.h>

#define READ 1 // 0001
#define WRITE 2 // 0010
#define EXE 4 // 0100
#define ALL_PERMISSION 7 // 0111

int main() {
    char file_permission = ALL_PERMISSION;
    file_permission &= ~READ & ~WRITE;

    if (file_permission & READ)
        printf("This file can be read.\n");
    if (file_permission & WRITE)
        printf("This file can be written.\n");
    if (file_permission & EXE)
        printf("This file can be executed.\n");
}
```

思考

- 如何添加权限?
- 如何切换权限?

```
#include <stdio.h>

#define READ 1 // 0001
#define WRITE 2 // 0010
#define EXE 4 // 0100
#define ALL_PERMISSION 7 // 0111

int main() {
    char file_permission = ALL_PERMISSION;
    file_permission &= ~READ & ~WRITE; // 删除权限
    file_permission |= WRITE | EXE; // 添加权限
    file_permission ^= READ; // 切换权限

    if (file_permission & READ)
        printf("This file can be read.\n");
    if (file_permission & WRITE)
        printf("This file can be written.\n");
    if (file_permission & EXE)
        printf("This file can be executed.\n");
}
```

调试 (Debug) 与错误 (Error)

- ❑ **调试 (Debug)** , 又称除错, 是发现和减少计算机程序错误的过程
- ❑ 程序错误的种类:
 - 语法 (编译/Compile) 错误, 这类错误编译器会给出提示, 如语句结束缺少 ";"
 - 运行时 (Runtime) 错误, 程序运行时异常退出, 给出或不给错误信息, 如段错误
 - 逻辑错误, 程序正常运行, 但未得到期望的结果
- ❑ 调试是发现**后两类错误**的重要手段

调试 (Debug) 方法

□ 调试方法一：

Where?

What?

- 使用 printf 函数在指定**位置**打印路径相关信息或**变量值**

□ 调试方法二：

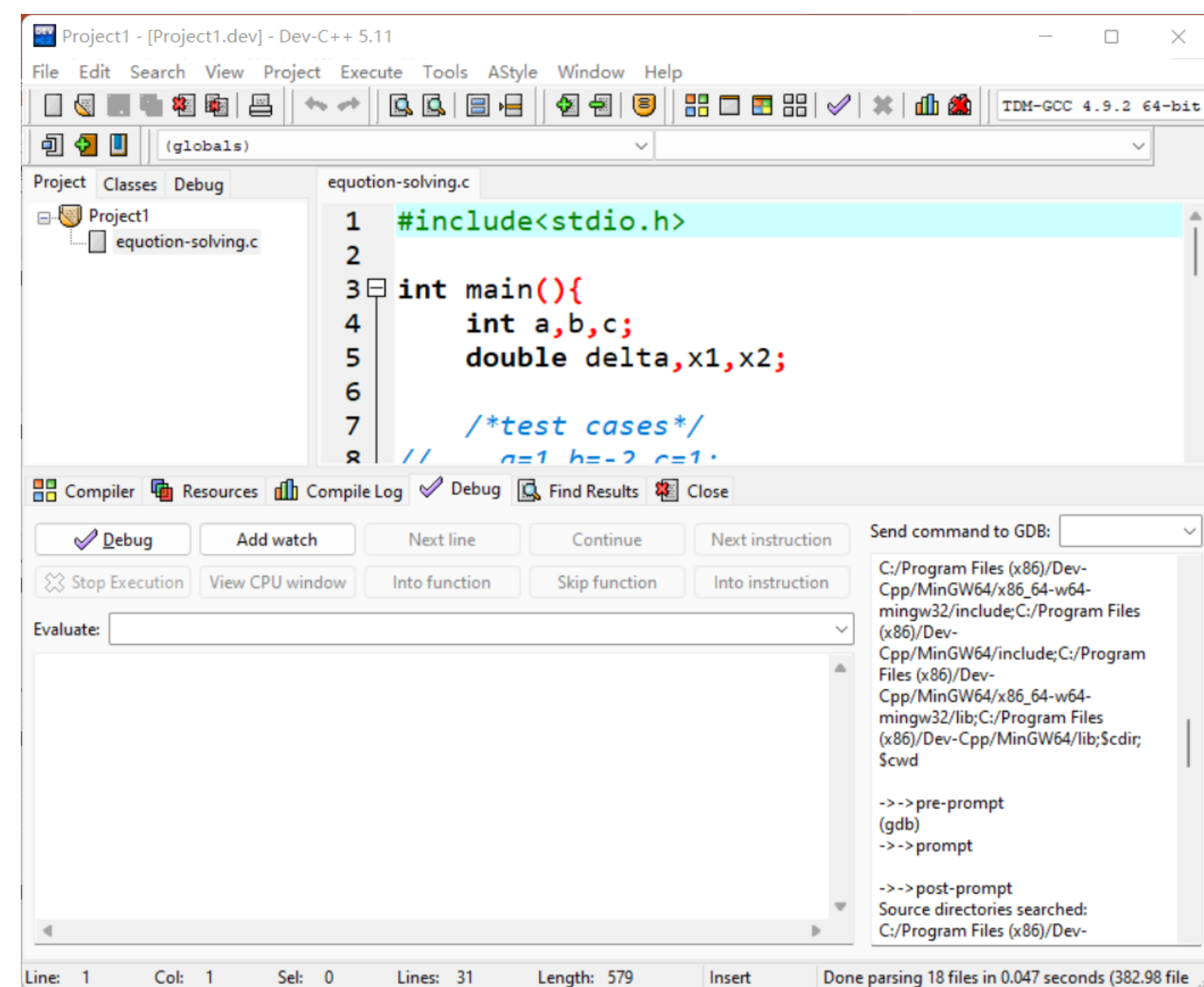
- **工具**，如 **DEV-C++ GDB** (GNU Debugger)
- **语句跟踪**，使用**断点 (Break Point)** 或执行下一句等指令跟踪程序执行路径
- **变量查看**，在跟踪的过程中观察变量的变化

□ 因此，需要掌握调试工具的使用以及调试的技巧

调试 (Debug) 工具: DEV C++

❑ 调试须知: 必须先创建项目再调试

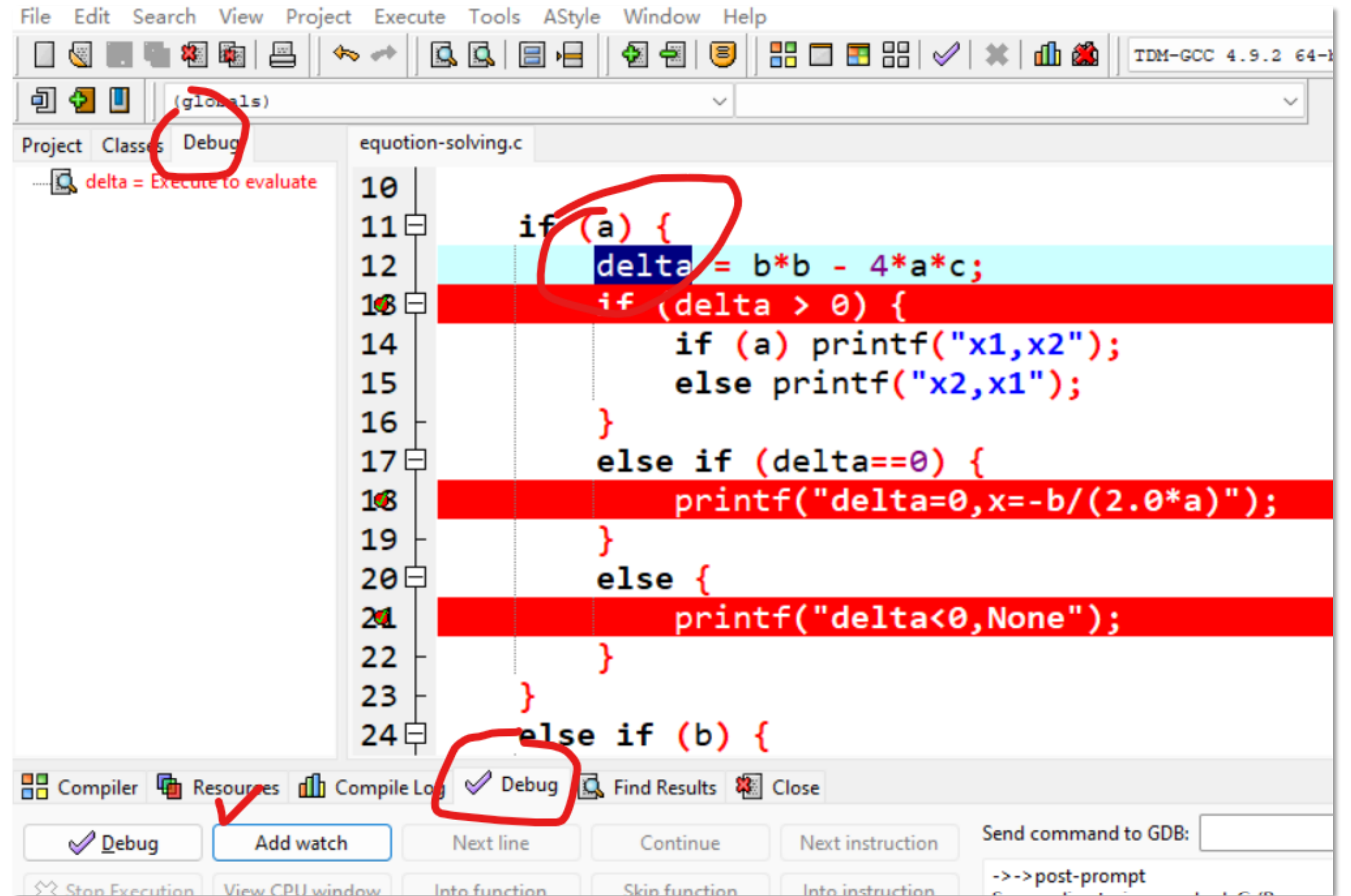
- 创建一个项目 (Project) , 选择 控制台应用 (Console Application)
- 将要调试 C 代码添加到该项目
- 点击代码, 打开代码
- 修改代码
- 按 F12 编译通过
- 按 F5 键开启调试



调试 (Debug) : 分支语句测试案例

□ 调试步骤:

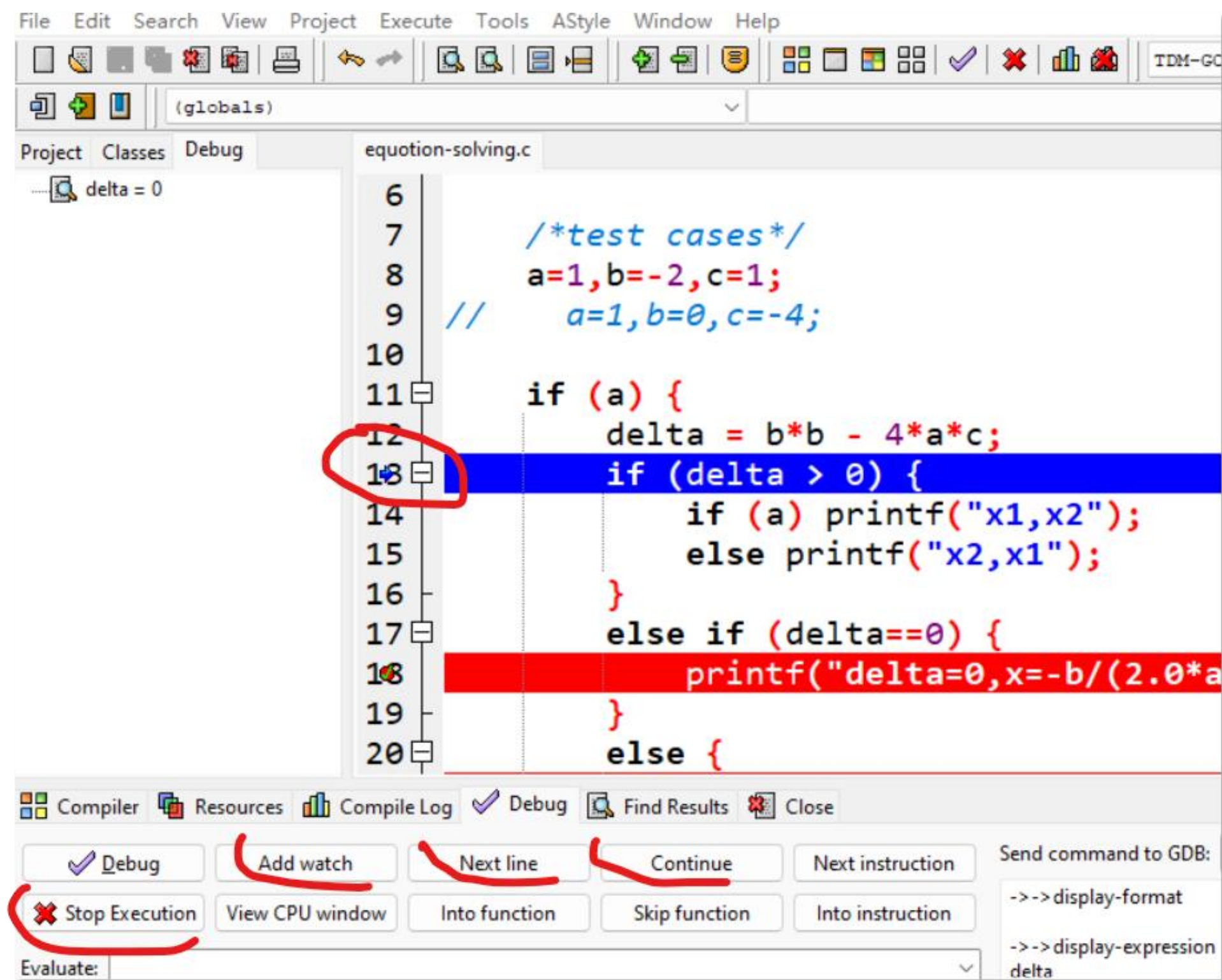
- 创建一个新项目
- 加入案例代码, equation-solving.c
- 按F12, 编译 (每次修改原代码后, 都需要重新编译)
- 设置断点: 点击 13, 18, 21行行号, 这些行变红
- 查看变量: 选中变量 delta, 按按钮 Add watch
- 按 F5



调试 (Debug) : 分支语句测试案例

□ 调试步骤:

- 这时程序会停在第一个断点 (蓝色行)
- 选中任何变量, 点 Add watch 观察变量值
- 点击行号, 添加/去除断点
- Next line 执行下一条语句
- Continue 执行到下一个断点或程序结束
- Stop Execution 调试结束



调试 (Debug) 过程

□ 调试前：

- 调试工具准备
- **设计测试案例 (数据点)**
- 修改测试程序

□ 调试中：

- 断点选择
- 变量查看
- 常用调试命令

调试 (Debug) : 代码修改

- 调试程序一般没有输入语句, 如 scanf
- 调试程序四段结构
 - 变量申明或定义
 - **测试案例**
 - 结果计算
 - 输出结果
- 程序四段结构
 - 变量申明或定义
 - **数据输入**
 - 结果计算
 - 输出结果
- 常用快捷键: ctrl+/ 触发注释代码, 在测试案例或数据输入间切换

调试 (Debug) : 分支语句测试案例

□ 测试案例是一组输入变量，它能覆盖所有可能的分支。例如

■ 求方程 $ax^2 + bx + c = 0$ 的实数解

■ 输入：三个整数 a, b, c , $|a|, |b|, |c| \in [0, 1000]$

□ 考虑 a, Δ 是否为零情况

■ 左边是分析

■ 红色每行是一个案例

■ 通常是最典型的输入

■ 易于观察各变量值

a	delta	b	a	b	c	期望输出
!=0	>0		1	0	-4	-2, 2
	=0		1	-2	1	1
	<0		1	2	2	None
=0	---	!=0	0	1	1	1
	---	0	0	0	1	None

调试 (Debug) : 分支语句练习

❑ 任务一：记录执行路径

- 断点设在11行，用 Next Line 执行到函数结束，记录执行完后经过的行号

❑ 任务二：拦截所有可能

- 断点设置在 if 或 else 后一行，思考为什么不要把语句与if, else写在一行上
- 修改测试案例，如 1 2 2，观察执行情况

❑ 任务三：检测分支

- 断点设置在 13 行，按 Next Line，请分析这个案例下 if 条件的结果

❑ 任务四：完善该程序代码

- 分别用设计的 6 个案例检测程序，完善程序并得到期望结果
- 最后用 ctrl+/ 屏蔽调式代码，给出数据输入代码

调试 (Debug) : 循环语句练习

- 用牛顿法计算平方根
 - 代码 newton.c 来源于网上, 经过适当修改
- 测试案例设计
 - 这里是 2 的平方根
- 断点选择和 Watch 变量
 - scanf 语句的下一句, 查看输入变量值
 - 循环调试断点一般选择3处:
 - while, for 关键字所在行,
 - 循环体的第一行
 - 循环语句的下一行 (已跳出循环)
 - Watch 是 条件表达式、关键变量
 - 使用 Next Line 跟踪一次迭代
 - 使用 Continue 跟踪每个迭代关键变量的变化, 观察迭代是否趋向结束 (收敛)

调试 (Debug)：循环语句练习

- ❑ 任务一：使用 Next Line 观察第一次迭代
- ❑ 任务二：使用 Continue 记录每次迭代后 x0 的变化，3-5个值
- ❑ 任务三：找到导致死循环的 Bug
- ❑ 任务四：完善程序，使它能计算你输入的数

项目1：剪刀石头布

- 剪刀石头布，对战计算机。采用 $2n+1$ 轮决定胜负：
 - 首先输入 n 决定比赛轮数，如 $n=1$ 则表示3轮2胜制
 - 每轮规则如下：
 - 计算机随机产生0-2数
 - 玩家输入 0, 1, 2 分别表示剪刀、石头和布
 - 石头砸剪刀、布包石头、剪刀裁布则玩家胜本轮，否则计算机胜
 - 输出本轮胜者，累计获胜次数
 - 最先达到胜 $(n+1)$ 次者获得最终胜利
- 请用 C 语言实现该游戏

谢谢大家！

邮箱: yuesh5@mail.sysu.edu.cn